User Acceptance Testing (UAT) Documentation

IITIIM Job Assistant Platform

Milestone 4: Admin Login & Approval Workflow

1. Description & Goal

Provide administrators with a console to review visitor registrations, view analytics, manage candidate subscription tiers, and audit administrative actions. Access boundaries are protected using Role-Based Access Control (RBAC).

2. Codebase Mapping

Flask Authentication & RBAC

- File: app.py
- Route @app.route("/admin/login"): Authenticates admin accounts by comparing passwords using SHA-256. Generates a signed JWT token with a 24-hour expiration duration containing admin_id, email, and role.
- admin_required Decorator: Middleware that intercepts admin API routes, checks for a valid Bearer JWT token in the Authorization header, decodes it with ADMIN_JWT_SECRET, and populates request.admin with database details.
- require_role Decorator: Enforces granular permissions (e.g., restricting administrative updates to SUPER_ADMIN and ADMIN roles, and blocking read-only USER accounts).

Admin Management API Routes

- File: app.py
- @app.route("/admin/verifications/<int:verif_id>/approve"): Approves a pending user registration and logs the action in the audit table.
- @app.route("/admin/verifications/<int:verif_id>/reject"): Rejects a pending registration. Requires a rejection reason, updates the account state, and logs the action.
- @app.route("/admin/dashboard"): Computes user counts, verified/pending/rejected metrics, active subscriptions, and conversion rates.
- @app.route("/admin/audit-logs"): Exposes a paginated search and filter endpoint for administrative actions.

Database Operations

- File: database.py
- Contains the underlying queries for auditing, verification approvals/rejections, and analytics dashboard computations.

Frontend Console Layout

- File: IITIIMJobAssistant_Admin_v3.html
- Provides the administration page layout. Contains a login card view (#view-login) and the main workspace app shell (#app-shell) with navigation sub-views: Dashboard, Approval Queue, User Directory, and Audit Log. Uses Chart.js for rendering signup trend charts.

Console Client Script

- File: admin.js
- Handles sign-in, session storage (sessionStorage caching of JWT token), state routing between sub-views, paginated tables, search triggers, filtering, approval/rejection triggers, and toast notices.

Console Design Stylesheet

- File: admin.css
- Features a responsive layout utilizing variables, a modern font stack, side navigation menus, glassmorphic header panels, custom tables, grid elements, badging components, status dot elements, and animated transitions.

3. Database Schema (users.db — Admin & Audit tables)

```
CREATE TABLE IF NOT EXISTS admin_users (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    email         TEXT    NOT NULL UNIQUE,
    password_hash TEXT    NOT NULL,
    role          TEXT    NOT NULL DEFAULT 'ADMIN',  -- SUPER_ADMIN | ADMIN | USER
    name          TEXT    NOT NULL DEFAULT '',
    created_at    TEXT    NOT NULL,
    updated_at    TEXT    NOT NULL,
    last_login_at TEXT    DEFAULT ''
);

CREATE TABLE IF NOT EXISTS audit_logs (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    admin_id        INTEGER NOT NULL,
    admin_email     TEXT    NOT NULL DEFAULT '',
    action          TEXT    NOT NULL,  -- VERIFICATION_APPROVED |
VERIFICATION_REJECTED | ADMIN_LOGIN
    target_user_id  INTEGER,
    previous_value  TEXT    DEFAULT '',
    new_value       TEXT    DEFAULT '',
    reason          TEXT    DEFAULT '',
    timestamp       TEXT    NOT NULL,
    ip_address      TEXT    DEFAULT ''
);
```

4. Verification & Testing Procedure

Admin Sign In

1. Start the Flask application. Navigate to http://localhost:5000/admin.
1. Attempt login with incorrect credentials. Verify the "Invalid email or password" error banner.
1. Login with valid admin credentials. Verify you are redirected to the dashboard, and admin_token is saved in session storage.

Dashboard Analytics Validation

1. Review the stat cards. Ensure the values match real registration statuses in users.db. Verify the registration line graph renders.

Approval Queue Verification

1. Navigate to Approval Queue. Ensure pending users are listed with their LinkedIn URLs.
1. Click Approve on a user. Confirm the prompt. Verify the card vanishes and the user's status in users.db updates to approved.
2. Click Reject on a user. Verify the modal opens and requires a reason. Select a reason, add notes, and submit. Verify the card vanishes, status updates to rejected, and reject_reason is stored.

Audit Trail Audit

2. Navigate to Audit Log.
2. Verify there are logs matching your login, approval, and rejection actions, including the IP address and change details.

RBAC Check

2. Attempt an admin action (e.g., approving a user) by sending a request to the API without the JWT token, or with an expired token. Verify the request is rejected with 401 Unauthorized.
2. Login as an account with USER role. Attempt to trigger approval via the console. Verify that the action fails and returns 403 Forbidden.